Onboarding a New Runtime Project

How to point ADOR at a new target project on **Windows 11** or **Ubuntu 22+**. The guided `ador init` wizard does most of this for you - it detects what is present and prints these same instructions in-line for anything missing. This guide is the reference for the non-ADOR prerequisites (the steps ADOR cannot do for you: installing a CLI, creating a token, choosing a runtime) plus the full prompt list.

> One ADOR install serves many projects. Each project you onboard becomes a named **profile**; use it with `ador --profile <name> ...` or make it active with `ador profile use <name>`.

---

0. Prerequisites (the non-ADOR steps)

ADOR's own CLI is stdlib-only Python, but it drives external tools. Install and authenticate these once per host. The wizard reports which are missing and prints the matching command.

Git

- **Windows:** `winget install Git.Git` (or https://git-scm.com/download/win).
- **Ubuntu:** `sudo apt-get update && sudo apt-get install -y git`.

A container runtime (docker or podman)

ADOR runs each ticket in an isolated container.

- **Ubuntu (native, recommended there):** Docker Engine (https://docs.docker.com/engine/install/ubuntu/) **or** podman (`sudo apt-get install -y podman uidmap slirp4netns`). Rootless podman needs `run_args = ["--userns=keep-id"]` (the wizard adds this automatically).
- **Windows:** enable WSL2 (`wsl --install`), install a distro (e.g. Ubuntu-24.04), then install Docker Engine or podman **inside the distro** (no Docker Desktop required) and use the runtime string `wsl -d Ubuntu-24.04 -- docker` (or `... -- podman`). ADOR handles the mount translation and env propagation across the WSL boundary.

The Git-platform CLI (glab for GitLab, gh for GitHub)

- **glab (GitLab):**
  - Windows: `winget install GLab.GLab` (or `scoop install glab`).
  - Ubuntu: download the `.deb` from the glab releases page and `sudo apt-get install ./glab_*.deb` (or `sudo apt-get install glab` where packaged).
- **gh (GitHub):**
  - Windows: `winget install GitHub.cli`.
  - Ubuntu: see https://github.com/cli/cli/blob/trunk/docs/install_linux.md .

Authenticate the platform CLI (create a token)

A **GitLab Personal Access Token**: GitLab → your avatar → *Preferences* → *Access Tokens* → *Add new token*; scope `api`; role **Developer** (Developer cannot merge protected branches - intended, ADOR never merges).

Copy it now (it is shown once). *A GitLab PAT looks like* `glpat-XXXXXXXXXXXXXXXXXXXX` *(the literal prefix* `glpat-` *followed by 20 characters).* Then: `glab auth login` (for a corporate/self-hosted instance choose the hostname, e.g. `gitlab.example.com`, and paste the token).

For **GitHub**: create a fine-grained PAT with Contents + Pull requests + Issues (read/write), then `gh auth login`.

Alternatively, put the token in the project `.env` as `GITLAB_TOKEN=` / `GITHUB_TOKEN=`. **ADOR never stores the token itself** - a profile records only the env-key *name*.

The executor model provider

The configured executor needs a model credential. In production, set `OPENAI_API_KEY` / `ANTHROPIC_API_KEY` in a `.env` beside the clone, or use the provider key required by the selected executor/model. The crawl-pilot host-auth OAuth bridge is not for unattended use; see `Setup_And_Credentials.md` section 3.1.

Build the executor images (once)

`docker/README.md` has the exact build commands (and the WSL2 variant). Build `ador-executor-base:0.2.0` for the generic executor environment, then build one runtime-project image with the target project's dependencies, for example the bundled `ador-executor-cockpit-gcp:latest` image or a project-specific tag.

---

1. Run the wizard

```
ador init sample-project                # interactive
ador init sample-project --answers a.toml   # non-interactive (CI/servers)
ador init sample-project --force            # overwrite an existing profile
```

The wizard walks these phases; each prompt shows a description, an example, a default, and validates your answer (re-prompting on error). Fixed-choice questions are shown as numbered options; type the number or the literal choice text.

Phase	You provide
A. Environment	(auto) a report + guidance for anything missing
B. Identity	profile name, one-line description, bot name (default ADOR)
C. Platform & repo	GitLab/GitHub; public or corporate (+ host URL); repo visibility; `group/sub/project` path
D. Local clone	absolute path to the clone; worktree root; target/base branch
E. Branch rules	whether a push rule exists; its regex; the branch template (checked against the rule)
F. Runtime & executor	container runtime (docker/podman/WSL-wrapped); image; executor CLI + model; auth mode

Phase	You provide
G. Delivery	reviewer username(s) for the draft MR
H. Budgets & policy	recommended vs custom budgets; run mode; where state lives
I. Review	confirm; the config is written, validated, and next steps printed

Non-interactive `--answers` files use the field keys as TOML keys, e.g.:

```
kind = "gitlab"
repo = "grp/sub/project"
repo_path = "/home/you/clone"        # or "D:/projects/clone" on Windows
runtime = "wsl -d Ubuntu-24.04 -- podman"
reviewers = "reviewer.one, reviewer.two"
has_push_rule = "yes"
branch_rule = "^(feature|bugfix|release|hotfix)/.*$"
branch_template = "feature/{slug}-{iid}"
confirm = "yes"
```

---

2. Use the profile

```
ador profile list                       # profiles + run stats, active marked *
ador profile show sample-project           # full metadata + statistics
ador profile use sample-project            # make it the active profile
ador --profile sample-project config validate
ador --profile sample-project run --dry-run
ador --profile sample-project run --once   # once glab is authenticated + image
built
```

Config resolution order: explicit `-c FILE` > `--profile NAME` > the active profile > `./config.toml`.

---

3. Verified

The assisted-config flow is live-verified on Windows 11 (native, `%APPDATA%` home) and Ubuntu 24.04/WSL2 (POSIX, `~/.config/ador`) - see `Prerequisites_Check_Report.md` section 17.